Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Parallel Programming
Assignment 5: Divide and Conquer
Spring Semester 2026

Assigned on: **18.03.2026**

Due by: **(Wednesday Exercise) 23.03.2026**
(Friday Exercise) 25.03.2026

Overview

In this assignment we will implement a multithreaded version of a sequential algorithm by using the Divide and Conquer design pattern. Furthermore, we review basic theoretical concepts of parallel programming including Amdahl's Law and Gustafson's Law.

Getting Prepared

- Fetch the new assignment from Gitlab by running:

  ```
  git fetch upstream
  git merge upstream/main
  ```

  You should now see the new assignment appear in your local directory.

- Open the project in IntelliJ IDEA: Click on *File* in the top-menu, then select *Open*. Select the folder of the new assignment.

1 Search and Count

When hiring new people for a job position, the first task is the screening of CVs. The Human Resources (HR) manager has to read all the CVs which have been previously validated by the automatic CV checker. The manager checks whether the applicants have a specific set of required qualities. If the applicant is qualified, then they mark those CVs as being valid for a certain job. However, some features are easier to check than others. For example, it is easier to get the age of the candidate based on the birth year, than to evaluate if the candidate has the necessary technical and social skills a job requires. We can say that the first task requires a *light* workload for each candidate, while the second task requires a *heavy* workload.

In our case, the CVs for the HR manager are randomly generated numbers and the desired features can be faster or slower to compute. We classify two types of workloads: `HEAVY` and `LIGHT` (e.g. check if the number is non zero - light workload, and check if the number is prime - heavy workload). The code to check the features is given to you and should not be changed (it is located in the class `Workload` in the `doWork` function). The task is to count for each feature how often it appears in the given input.

The HR manager knew about the advantages of automating the CV screening process and bought a single threaded program to solve this task. The relevant code is located in the `SearchAndCountSeq` class. But with a rising number of applicants the HR manager hopes that the program can be made faster.

Task A: The HR manager hired you to speed-up the screening process by creating a parallel version of the program. As a first step, you need to decide how to split the problem into smaller tasks to be solved in parallel. For this purpose rewrite the sequential version using the Divide and Conquer design pattern:

```
Divide and Conquer:
  if cannot divide:
    return unitary solution (stop recursion)
  divide problem into two
  solve first (recursively)
  solve second (recursively)
  return combine solutions
```

Complete the sequential implementation using the Divide and Conquer design pattern in the provided skeleton class `SearchAndCountSeqDivideAndConquer`. Make sure that your implementation is correct.

Task B: Solving our problem using the Divide and Conquer design pattern gives us a straightforward way to make it parallel – instead of executing the two tasks sequentially we execute them in parallel. Implement a parallel version of `SearchAndCountSeqDivideAndConquer` in the provided skeleton class `SearchAndCountThreadDivideAndConquer`.

Extend your implementation such that it creates only a fixed number of threads regardless of the problem size. Make sure that your solution is properly synchronized when checking whether to create a new thread.

Task C: Next, the HR manager wants you to show them that your program indeed works faster. Compare the runtime of your parallel implementation with the original sequential version. To improve the runtime of your parallel implementation implement a sequential `cutOff`, that is, use the sequential version to process inputs smaller than the `cutOff` value instead of recursing always to the base case that is not divisible (i.e., processing a single element which is equivalent to the `cutOff` value 1).

Further, your manager does not want to meddle with its settings. Find the value for the `cutOff`, and the number of threads that maximize the speedup for input size $100,000$ and for both workloads.

Task D (Optional): Instead of creating threads manually, adapt your code to use `ExecutorService`.

2 Amdahl's and Gustafson's Law

Assuming a program consists of 50% non-parallelizable code.

a) Compute the speed-up when using 2 and 4 processors according to Amdahl's law.

b) Now assume that the parallel work per processor is fixed. Compute the speed-up when using 2 and 4 processors according to Gustafson's law.

c) Explain why both speed-up results are different.

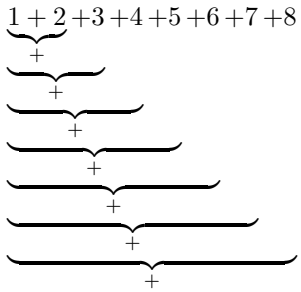
3 Amdahl's and Gustafson's Law II

a) The analysis of a program has shown a speedup of 3 when running on 4 cores. What is the serial fraction according to Gustafson's law?

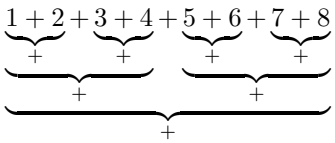
b) The analysis of a program has shown a speedup of 3 when running on 4 cores. What is the serial fraction according to Amdahl's law (assuming best possible speedup)?

4 Task Graph

Assuming you want to add eight numbers, then two options to do this are



and



a) Given those two variants, determine the length of the critical path for both computations.

b) For a sequence of length n, determine the length of the critical path using the two approaches from above (accumulator method and Divide and Conquer).

Submission

You need to submit your code and reports by using our submission system. This step will be the same for all of the subsequent exercises, but we will explain it in detail here so you can familiarize yourself with the system.

To submit your code using the terminal, follow the instructions below:

- In the terminal, navigate to your personal repository.
- Add your changes:
  ```
  git add .  # this command adds all changes in the git repo
  ```
- Commit your changes. In your commit message, make sure to include the [Submission] tag.
  ```
  git commit -m "[Submission] ..."
  ```
- Push your changes!
  ```
  git push
  ```

To submit your code using IntelliJ, follow the instructions below:

- Press the "Commit" button (found on the left hand toolbar)
- Select all the files you want changed during the assignment.
- Write a descriptive commit message. Make sure to include the [Submission] tag.
- Press "Commit and Push"

After this step, you can check your repository on the GitLab website in your browser. Make sure that you see your changes there!

Troubleshooting

Git Issues
In rare cases, you might get an error like this:

```
Can't connect to any URI
https://gitlab.inf.ethz.ch/COURSE-PPROG26/pprog26-<username >.git
(https://gitlab.inf.ethz.ch/COURSE-PPROG26/pprog26-<username>.git:
https://gitlab.inf.ethz.ch/COURSE-PPROG26/pprog26-<username >.git/info/refs?
service=git-receive-pack not found)
```

It means that no repository was created for your username yet. If that is the case, please contact your teaching assistant. The teaching assistant will be able to create the repository location for you. After that, you should be able to submit your work by following the instructions above.

IntelliJ / Java Issues
If you have trouble running the Java programs in IntelliJ, check the following:

- You have marked the `src` directory as a "Source Root". Right click on the folder `src`, select "Mark Directory As >Source Root".
- You have installed an updated JDK. We recommend `openjdk-25`. You can check this under "File >Project Structure >SDK"